

Chuleta de métodos en Java

public · static · void · class — cuándo usar cada cosa

1. Palabras clave, una a una

class

El molde/plano de un objeto. Toda clase que declaras necesita esta palabra. No es opcional ni depende de nada.

public / private

¿Quién puede acceder? public = cualquiera desde fuera. private = solo la propia clase. Los atributos suelen ser private para proteger los datos.

static

¿Pertenece a la clase o a un objeto? Con static se llama directo: Clase.metodo(). Sin static hace falta 'new Clase()' primero. Tip: si lo llamas desde el main, tu método también debe ser static.

void / tipo

¿Devuelve algo? void = no devuelve nada (solo actúa). Si pones int, String, double... el método DEBE terminar con 'return valor;' de ese mismo tipo.

2. Las 4 combinaciones más comunes de un método

static void metodo()

No devuelve nada y no necesita datos de fuera.

```
static void saludar() {  
    System.out.println("Hola");  
}
```

static void metodo(int x)

No devuelve nada, pero necesita datos para trabajar.

```
static void mostrarDoble(int x) {  
    System.out.println(x * 2);  
}
```

static int metodo()

Devuelve un valor calculado sin necesitar datos externos.

```
static int numeroFijo() { return 10; }
```

static int metodo(int[] a)

Devuelve un valor Y necesita datos para calcularlo.

```
static int sumar(int[] a) { int s = 0; for (int  
n : a) s += n; return s; }
```

3. La idea que resuelve el 90% de las dudas

El tipo que escribes ANTES del nombre del método es una promesa: dice qué va a devolver.

- Si pones **void** → el método no debe tener ningún **return valor;** dentro (como mucho un **return;** vacío para cortar antes de tiempo).
- Si pones **int**, **String**, **double**... → el método DEBE terminar (en todos los caminos posibles) con un **return** que entregue un valor de ese tipo.

Truco para decidir rápido: pregúntate '¿voy a necesitar usar el resultado de esto después, guardándolo en una variable?' Si sí → necesita tipo de retorno. Si solo necesitas que 'haga algo' (imprimir, modificar) → void.

4. El caso especial: public static void main(String[] args)